



ChatGPT Cheat Sheet

For Data Science

Learn Data Science Online at
www.DataCamp.com



How to use this cheatsheet?

1

What is ChatGPT?

ChatGPT is a language model developed by OpenAI that has gained significant attention due to its remarkable ability to generate human-like responses to prompts given to it. ChatGPT is useful for a variety of language-based tasks, including language translation, question answering, text completion, and a lot more. It's also very useful for **data science workflows**.

2

How to get started?

To get started, simply head to OpenAI's site and start inputting prompts. Register for free or upgrade to a paid version for priority access.

<https://chat.openai.com/chat>

3

What to expect in this cheat sheet?

In this cheat sheet, you will find specific examples of using ChatGPT for common data science tasks, including data visualization, data analysis, machine learning, and more.



Our guide for great prompts

1

Start prompts with a description of the AI's role. For example, “You are a {x}” or “I want you to act as {x}”. If you aren't sure, try “You are a helpful assistant”.
e.g., You are a data scientist at OpenAI, you are working on large language models. Explain to me how GPT-3 works.

2

Use it for common tasks. The more common the task, the better the chances of success with ChatGPT.

3

Offer context. Offering context when posing a question to ChatGPT allows it to provide specific answers. Providing context allows the model to understand the nuances of the problem and generate more informed responses.
e.g., You are a data scientist at OpenAI, you are working on large language models. Explain how GPT-3 works to a business executive with no technical background.

4

Be precise. Being precise is another best practice that can minimize the need for tweaking the output. When posing a question, be clear about the input (if any) and the desired outcome.

5

Keep trying. Do not be afraid to try multiple prompts. Using different prompts can provide different perspectives on a problem and enable the model to generate a variety of responses.



Limitations of ChatGPT

However, ChatGPT is not perfect – it can generate wrong information, or “hallucinate”. ChatGPT can provide incorrect answers in an authoritative tone. As such, it’s very important to fact check all of its answers.

k

which is heavier, 10kg iron or 10kg cotton?



10kg iron is heavier than 10kg cotton. Iron is typically heavier than cotton, so the answer is 10kg iron is heavier. Answer: \boxed{iron}.



[Source: Gary Marcus](#)

Prompts for general coding workflows



Debugging Python code

PROMPT STRUCTURE

I want you to be a Python programmer, here is a piece of Python code containing {problem} — {insert code snippet} — I am getting the following error {insert error}. What is the reason for the bug?



I want you to be a Python programmer, here is a piece of python code containing a print function, `print(Hello World)`, I am getting the following error `SyntaxError: invalid syntax`. What is the reason for the bug?



The Python code you provided is missing quotation marks around the string "Hello World", which is causing the `SyntaxError`.  

To fix the error, simply add quotation marks around the string, like this:

python

 Copy code

```
print("Hello World")
```

The corrected code will print "Hello World" to the console without any errors.



Debugging R code

PROMPT STRUCTURE

I want you to be an R programmer, here is a piece of R code containing {problem} — {insert code snippet} — I am getting the following error {insert error}. What is the reason for the bug?



I want you to be R Programmer, here is a piece of R code containing a print function—
`foo <- list(str='R',vec=c(2,3,4),bool=TRUE)`
`if( foo['bool'] ) {print('Hi!')}` — I am getting the following error unexpected end of input.
What is the reason for the bug?



The reason for the error is that the square brackets (``[ ]``) are used to extract elements from a list in R, but they return a new list rather than a single value when used with a single index. So, when ``foo['bool']`` is evaluated, it returns a new list that contains the element ``'bool'`` of the original list ``foo``.



The ``if`` statement expects a single logical value as its argument, but instead it receives a list as returned by ``foo['bool']``. Therefore, the ``unexpected end of input`` error occurs because R is expecting a logical expression after the ``if`` statement.

To fix the error, you can extract the value of the ``'bool'`` element from the list using double square brackets (``[[ ]]``) instead of single square brackets, which will return the logical value of ``TRUE`` in this case:

lua

Copy code

```
foo <- list(str='R', vec=c(2, 3, 4), bool=TRUE)
if (foo[['bool']]) {
  print('Hi!')
}
```



Debugging SQL code

PROMPT STRUCTURE

I want you to be a SQL programmer, here is a piece of SQL code containing {problem} — {insert code snippet} — I am getting the following error {insert error}. What is the reason for the bug?



I want you to be SQL Programmer, here is a piece of SQL snippet containing a print function—SELECT customer_id FROM sales WHERE COUNT(customer_id)>5 — I am getting the following error "misuse of aggregate function COUNT()". What is the reason for the bug?



The reason for the error "misuse of aggregate function COUNT()" is that the COUNT function is an aggregate function in SQL and cannot be used in the WHERE clause without a GROUP BY clause.



In this case, the query is trying to filter records where the COUNT of customer_id is greater than 5. Instead, the query should use a HAVING clause to filter records based on the COUNT result after grouping the records by customer_id.

Here's an example of how the query should be written:

sql

Copy code

```
SELECT customer_id, COUNT(customer_id) as customer_count
FROM sales
GROUP BY customer_id
HAVING COUNT(customer_id) > 5;
```

This will group the records by customer_id, count the number of records for each customer, and then filter out the records where the count is less than or equal to 5.



Python code explanation

PROMPT STRUCTURE

I want you to act as a code explainer in Python. I don't understand this function. Can you please explain what it does, and provide an example? {Insert function}



I want you to act as a code explainer in python. I don't understand this function. Can you please explain what it does, and provide an example? 

```
def spin_words(sentence):  
    # Your code goes here  
    return " ".join([x[::-1] if len(x) >= 5 else x for x in sentence.split(" ")])
```



Sure, I'd be happy to explain this function to you!  

This function, `spin_words`, takes a string `sentence` as input and returns a new string that has all the words with length 5 or more reversed.

Here's how the function works:

1. `sentence.split(" ")` splits the input string `sentence` into a list of words, based on the space character.
2. The list comprehension `[x[::-1] if len(x) >= 5 else x for x in sentence.split(" ")]` iterates over each word in the list, and applies the following logic:
 - If the length of the word `x` is greater than or equal to 5, then reverse the word using the `[::-1]` syntax, which is a slice that starts from the end of the string and goes backwards to the beginning.
 - If the length of the word `x` is less than 5, then return the word as-is.
3. `" ".join()` combines the list of words into a single string, with each word separated by a space character.
4. The resulting string is returned as the output of the function.



R code explanation

PROMPT STRUCTURE

I want you to act as a code explainer in R. I don't understand this function. Can you please explain what it does, and provide an example? {Insert function}



I want you to act as a code explainer in R. I don't understand this function. Can you please explain what it does, and provide an example?

```
rec_fac <- function(x){  
  if(x==0 || x==1){  
    return(1)  
  } else {  
    return(x*rec_fac(x-1))  
  }  
}
```



Certainly, I'd be happy to explain this function to you!

This function, `rec_fac`, is a recursive function in R that computes the factorial of a given number `x`.

Here's how the function works:

1. The function takes an input argument `x`.
2. The function checks if `x` is equal to 0 or 1. If `x` is 0 or 1, the function returns 1, because the factorial of 0 or 1 is always 1.
3. If `x` is not 0 or 1, the function calls itself recursively with the argument `x-1`. This means that the function will keep calling itself, with the argument decreasing by 1 each time, until it reaches the base case where `x` is 0 or 1.
4. Once the base case is reached, the recursive function returns 1 and the factorial calculation continues.
5. When the recursive function has completed all the recursive calls and returned the base case value of 1, the multiplication of all the intermediate values will be computed to produce the final factorial value.
6. The final computed value is returned as the output of the function.



SQL code explanation

PROMPT STRUCTURE

I want you to act as a code explainer in SQL. I don't understand this snippet. Can you please explain what it does, and provide an example?
{Insert SQL query}



I want you to act as a code explainer in SQL. I don't understand this snippet. Can you please explain what it does, and provide an example?



```
SELECT start_terminal,
       duration_seconds,
       SUM(duration_seconds) OVER
         (PARTITION BY start_terminal ORDER BY start_time)
       AS running_total
FROM dc_bikeshare_q1_2012
```



Certainly, I'd be happy to explain this SQL snippet!



This code is a SQL query that performs a window function to calculate the running total of the `duration_seconds` column, partitioned by `start_terminal` and ordered by `start_time`. Here's what each part of the query does:

- `SELECT start_terminal, duration_seconds`: This selects the `start_terminal` and `duration_seconds` columns from the `dc_bikeshare_q1_2012` table.
- `SUM(duration_seconds) OVER`: This sets up a window function that will calculate the running total of the `duration_seconds` column.
- `(PARTITION BY start_terminal ORDER BY start_time)`: This defines the partitioning and ordering of the window function. The `PARTITION BY` clause specifies that the running total should be calculated separately for each unique value of `start_terminal`. The `ORDER BY` clause specifies that the rows within each partition should be ordered by `start_time`.
- `AS running_total`: This gives a name to the running total column that will be returned by the query.

An example of how this query might be used is as follows:



Python code optimization

PROMPT STRUCTURE

I want you to act as a code optimizer in Python. {Describe problem with current code, if possible}. Can you make the code {more Pythonic/cleaner/more efficient/run faster/more readable}? {Insert Code}



I want you to act as a code optimizer in Python. Can you make the code more pythonic? 

```
>>> input_list = [1, 2, -3]
>>> output_list = []
>>> for x in input_list:
...     if x >= 0:
...         output_list.append(1)
...     else:
...         output_list.append(0)
>>> output_list
```



Sure! Here's a more pythonic way to write the code:  

makefile

 Copy code

```
input_list = [1, 2, -3]
output_list = [1 if x >= 0 else 0 for x in input_list]
```

In this code, we are using a list comprehension to create the `output_list` in a single line. The expression `[1 if x >= 0 else 0 for x in input_list]` generates a new list by iterating over the elements of `input_list`. For each element `x`, it checks if `x >= 0`. If it is, it adds `1` to the new list, otherwise it adds `0`.

This code is more concise, easier to read, and follows Python's style guidelines.



R code optimization

PROMPT STRUCTURE

I want you to act as a code optimizer in R. {Describe problem with current code, if possible}. Can you make the code {cleaner/more efficient/run faster/more readable}? {Insert Code}



I want you to act as a code optimizer in R. Can you make the code faster?



```
v <- c(1,4,5,3,54,6,7,5,3,5,6,4,3,4,5)
```

```
for(i in 1:length(v)){  
  v[i] <- v[i] + 42  
}
```

```
v
```



Sure! Here's a way to make the code faster:



```
r
```

Copy code

```
v <- c(1,4,5,3,54,6,7,5,3,5,6,4,3,4,5)  
v <- v + 42
```

Instead of iterating over the elements of the vector `v` with a for loop, we can directly add 42 to the vector using the addition operator `+`. This way, we perform the operation on the entire vector at once, which is generally faster than iterating over the elements of the vector one by one.

This code is simpler, more efficient, and easier to read.



SQL code optimization

PROMPT STRUCTURE

I want you to act as a query optimizer in SQL. {Describe problem with current code, if possible}. Can you suggest ways to make the query {run faster/more readable/simpler}? {Insert Code}



I want you to act as a query optimizer in SQL. The following query takes very long to run. Can you suggest ways to make this query run faster?

```
SELECT DISTINCT Customer.CustId
, Customer.Name
, Customer.EMailAddr
FROM Customer
WHERE EMailAddr ILIKE '%GMAIL%' OR EMailAddr ILIKE '%YAHOO%'
```



Sure! Here are some suggestions to make the query run faster:

1. Use an index: If you have an index on the `EMailAddr` column, the database can use it to speed up the search for email addresses containing "GMAIL" or "YAHOO". If you don't have an index, consider adding one.
2. Use UNION instead of OR: In some cases, using `UNION` can be faster than using `OR`. You can split the query into two separate queries, one for email addresses containing "GMAIL" and one for email addresses containing "YAHOO". Then, you can combine the results using `UNION`.

Here's an example of how to use `UNION`:

```
sql
SELECT DISTINCT CustId, Name, EMailAddr
FROM Customer
WHERE EMailAddr ILIKE '%GMAIL%'
UNION
SELECT DISTINCT CustId, Name, EMailAddr
FROM Customer
WHERE EMailAddr ILIKE '%YAHOO%';
```

Copy code



Python code simplification

PROMPT STRUCTURE

I want you to act as a programmer in Python. Please simplify this code while ensuring that it is {efficient/easy to read/Pythonic}?
{Insert Code}



I want you to act as a programmer in Python. Please simplify this code while ensuring that it is easy to read.

```
decision = input("Would you like to try again?(y/n) (or 'Help' for instructions")
if decision != "y" or decision != "n" or decision != "Help":
    print("This input is not readable! Try again!")
```



Sure! Here's a simplified version of the code that is easier to read:

python Copy code

```
decision = input("Would you like to try again? (y/n) (or 'help' for instructions")
if decision not in ("y", "n", "help"):
    print("Input not recognized. Please try again.")
```

In this version, we check if the user input is in the expected list of values, which includes `y`, `n`, and `help`. If the input is not recognized, we print an error message asking the user to try again. We also added a colon to the end of the prompt to make it clearer that the user should provide an input after it.



R code simplification

PROMPT STRUCTURE

I want you to act as a programmer in R. Please simplify this code while ensuring that it is {efficient/easy to read}? {Insert Code}



I want you to act as a programmer in R. Please simplify this code while ensuring that it is easy to read.



```
vector = c(1,3,5,8,5,2,3,4,5,6,5,0,7,8,9)
new_vector <- ifelse(((vector==1)|(vector==3)|(vector==5)), "A", "B")
```



One way to simplify the code while making it easier to read is to use the `%in%` operator to check if values are present in a set of values instead of using multiple `==` operators. Here's the simplified code:



c

Copy code

```
vector = c(1,3,5,8,5,2,3,4,5,6,5,0,7,8,9)
new_vector <- ifelse(vector %in% c(1, 3, 5), "A", "B")
```

This code first creates a vector `vector`, and then uses the `ifelse` function to create a new vector `new\_vector` that contains "A" if the corresponding element of `vector` is 1, 3, or 5, and "B" otherwise. The `%in%` operator is used to check if the elements of `vector` are present in the set of values `c(1, 3, 5)`.



SQL code simplification

PROMPT STRUCTURE

I want you to act as a SQL programmer. I am running {PostgreSQL 14/MySQL 8/SQLite 3.4/other versions.}. Can you please simplify this query {while ensuring that it is efficient/easy to read/insert any additional requirements}?



I want you to act as a SQL code programmer. I am running PostgreSQL 14. Can you please simplify this query?

```
SELECT DISTINCT Customer.CustId
, Customer.Name
, Customer.EMailAddr
FROM Customer
WHERE
  EMailAddr IN (
    SELECT EMailAddr
    FROM Customer
    WHERE EMailAddr ILIKE '%GMAIL%' OR EMailAddr ILIKE '%YAHOO%'
    GROUP BY EMailAddr
    HAVING COUNT(*) > 1
  )
```



Sure! Here's a simplified version of the query:



sql

Copy code

```
SELECT DISTINCT CustId, Name, EMailAddr
FROM Customer
WHERE EMailAddr ILIKE '%GMAIL%' OR EMailAddr ILIKE '%YAHOO%'
GROUP BY CustId, Name, EMailAddr
HAVING COUNT(*) > 1;
```



From R to Python code translation

PROMPT STRUCTURE

I want you to act as a programmer in R. Please translate this code to Python. {Insert code}



I want you to act as a programmer in R. Please translate this code to python



```
vector = c(1,3,5,8,5,2,3,4,5,6,5,0,7,8,9)
new_vector <- ifelse(vector %in% c(1, 3, 5), "A", "B")
```



Here's the equivalent code in Python:



python

Copy code

```
vector = [1, 3, 5, 8, 5, 2, 3, 4, 5, 6, 5, 0, 7, 8, 9]
new_vector = ["A" if x in [1, 3, 5] else "B" for x in vector]
```

The `%in%` operator in R checks whether each element of a vector is a member of another vector. In Python, we can use the `in` operator to achieve the same result. The `ifelse` function in R returns a vector of the same length as its test condition, with elements taken from either of the two vectors depending on the outcome of the test. In Python, we can use a list comprehension to achieve a similar result.



From Python to R code translation

PROMPT STRUCTURE

I want you to act as a programmer in Python. Please translate this code to R. {Insert code}



I want you to act as a programmer in Python. Please translate this Python code to R.



```
decision = input("Would you like to try again? (y/n) (or 'help' for instructions): ")
if decision not in ("y", "n", "help"):
    print("Input not recognized. Please try again.")
```



The equivalent R code for the given Python code is:



R

Copy code

```
decision <- readline("Would you like to try again? (y/n) (or 'help' for inst
if (!decision %in% c("y", "n", "help")) {
  cat("Input not recognized. Please try again.")
}
```

Note that in R, `readline()` is used to read input from the console, and `cat()` is used to print output to the console. The `not in` operator in Python is equivalent to `! %in%` in R.



Compare function speeds in python

PROMPT STRUCTURE

I want you to act as a Python programmer. Can you write code that compares the speed of two functions {functionname} and {functionname}? {Insert functions}



I want you to act as a Python programmer. Can you write code that compares the speed of two functions func1 and func2?



```
def func1():
    input_list = [1, 2, -3]
    output_list = []
    for x in input_list:
        if x >= 0:
            output_list.append(1)
        else:
            output_list.append(0)
    return output_list
def func2(input_list):
    return [1 if x >= 0 else 0 for x in input_list]
```



Sure! Here's some sample code that uses the `timeit` module to time the execution of `func1` and `func2` and compare their speeds:



python

Copy code

```
import timeit

def func1():
    input_list = [1, 2, -3]
    output_list = []
    for x in input_list:
        if x >= 0:
            output_list.append(1)
        else:
            output_list.append(0)
    return output_list
```



Write unit tests in R

PROMPT STRUCTURE

I want you to act as a R Programmer. Can you please write unit tests for the function {functionname}? {Insert requirements for unit tests, if any} {Insert code}



I want you to act as a R programmer. Can you please write unit tests for the the findfactorial function?



```
findfactorial <- function(n){  
  
  factorial <- 1  
  
  if ((n==0)|(n==1))  
    factorial <- 1  
  
  else{  
    for( i in 1:n)  
      factorial <- factorial * i  
  }  
  return (factorial)  
}
```



Sure! Here are some unit tests for the `findfactorial` function in R using the `testthat` package:



scss

Copy code

```
library(testthat)  
  
# Test for n = 0  
test_that("findfactorial returns 1 when n = 0", {  
  expect_equal(findfactorial(0), 1)  
})  
  
# Test for n = 1  
test_that("findfactorial returns 1 when n = 1", {  
  expect_equal(findfactorial(1), 1)  
})
```



Write unit tests in Python

PROMPT STRUCTURE

I want you to act as a Python Programmer. Can you please write unit tests for the function {functionname}? {Insert requirements for unit tests, if any} {Insert code}



I want you to act as a Python programmer. Can you please write unit tests for the findfactorial function. Include tests for positive numbers, negative numbers, zero, and strings.



```
def findfactorial(x):  
    if x == 1:  
        return 1  
    else:  
        return (x * factorial(x-1))
```



Certainly! Here's an example test suite that includes tests for positive numbers, negative numbers, zero, and strings.



python

Copy code

```
import unittest  
  
class TestFindFactorial(unittest.TestCase):  
  
    def test_positive_numbers(self):  
        self.assertEqual(findfactorial(5), 120)  
        self.assertEqual(findfactorial(8), 40320)  
  
    def test_negative_numbers(self):  
        self.assertRaises(ValueError, findfactorial, -5)  
        self.assertRaises(ValueError, findfactorial, -8)  
  
    def test_zero(self):  
        self.assertEqual(findfactorial(0), 1)
```

Prompts for data analysis workflows



Data generation & creating tables

PROMPT STRUCTURE

I want you to act as a data generator. Can you write SQL queries in {database version} that create a table {table name} with the columns {column name}. Include relevant constraints and index.



I want you to act as a data generator. Can you write SQL queries in PostgreSQL 14 that create a table "customers" with the columns "customer_id", "username", "email", "join_date", "last_purchase_date". Include relevant constraints and index.



Sure! Here's an example SQL script that creates a table named `customers` with the columns you requested and includes relevant constraints and index:



sql

Copy code

```
CREATE TABLE customers (  
  customer_id SERIAL PRIMARY KEY,  
  username VARCHAR(50) NOT NULL,  
  email VARCHAR(100) NOT NULL,  
  join_date DATE NOT NULL,  
  last_purchase_date DATE,  
  CONSTRAINT email_unique UNIQUE (email),  
  CONSTRAINT username_unique UNIQUE (username)  
);  
  
CREATE INDEX idx_customers_join_date ON customers(join_date);  
CREATE INDEX idx_customers_last_purchase_date ON customers(last_purchase_date);
```

This script creates a `customers` table with a `customer\_id` column as a primary key, `username`, `email`, `join\_date`, and `last\_purchase\_date` columns as well. It also includes two unique constraints on the `email` and `username` columns to ensure that no two customers have the same email or username. Finally, it creates two indexes on the



Common table expressions

PROMPT STRUCTURE

I want you to act as a SQL code programmer. I am running {database version}. Can you rewrite this query using CTE? {Insert query}



I want you to act as a SQL code programmer. I am running PostgreSQL 14. Can you rewrite this query using CTE?

```
SELECT DISTINCT Customer.CustId
, Customer.Name
, Customer.EmailAddr
FROM Customer
WHERE
EmailAddr IN (
SELECT EmailAddr
FROM Customer
WHERE EmailAddr ILIKE '%GMAIL%' OR EmailAddr ILIKE '%YAHOO%'
GROUP BY EmailAddr
HAVING COUNT(*) > 1)
)
```



Sure! Here's an example of how to rewrite the query using a common table expression (CTE):



sql

Copy code

```
WITH email_counts AS (
SELECT EmailAddr
FROM Customer
WHERE EmailAddr ILIKE '%GMAIL%' OR EmailAddr ILIKE '%YAHOO%'
GROUP BY EmailAddr
HAVING COUNT(*) > 1
)
SELECT DISTINCT CustId, Name, EmailAddr
FROM Customer
WHERE EmailAddr IN (SELECT EmailAddr FROM email_counts);
```



Write SQL queries from natural language

PROMPT STRUCTURE

I want you to act as a data scientist. {Insert description of tables}. Can you {count/sum/take average} of {value} which are {insert filters}

Example: Data aggregation in SQL



I want you to act as a data scientist. I have three PostgreSQL 14 tables "customers" for an e-commerce company. The customers table consists of the columns "customer_id", "customer_name", "customer_email", "customer_phone", and "customer_address". Using "customer_email", can you count the number of customers who use gmail, outlook, yahoo, or other providers?



Sure! Here's a SQL query you can use to count the number of customers who use different email providers:



```
sql Copy code

SELECT
  CASE
    WHEN customer_email LIKE '%@gmail.com' THEN 'Gmail'
    WHEN customer_email LIKE '%@outlook.com' THEN 'Outlook'
    WHEN customer_email LIKE '%@yahoo.com' THEN 'Yahoo'
    ELSE 'Other'
  END AS email_provider,
  COUNT(*) AS num_customers
FROM customers
GROUP BY email_provider
ORDER BY num_customers DESC;
```

This query uses a CASE statement to categorize email addresses into four groups: "Gmail", "Outlook", "Yahoo", and "Other". It then counts the number of customers in each group and orders the results by the number of customers in descending order.

You can modify the email domain name (e.g., 'gmail.com') and add more email domains to this query to count the number of customers who use other email providers.



Write SQL queries from natural language

PROMPT STRUCTURE

I want you to act as a data scientist. I am running {PostgreSQL 14/MySQL 8/SQLite 3.4/other versions.}. I have the tables {table_name} which are {table description}. The sales table consists of the columns {column names}. Can you please write a query that finds the 7-day running average of {quantity}?

Example: 7 day running average in SQL



I want you to act as a data scientist. I have PostgreSQL 14 tables "sales" for an e-commerce company. The sales table consists of the columns "customer_id", "product_id", "sale_date", "sale_quantity". Can you please write a query that finds the 7-day running average of sale_quantity?



Sure, here's a query that calculates the 7-day running average of sale_quantity for each customer_id and product_id:



sql

Copy code

```
SELECT
  customer_id,
  product_id,
  sale_date,
  AVG(sale_quantity) OVER (
    PARTITION BY customer_id, product_id
    ORDER BY sale_date
    ROWS BETWEEN 6 PRECEDING AND CURRENT ROW
  ) AS running_avg
FROM
  sales;
```

Explanation:

- The `AVG()` function calculates the average sale_quantity over a window of 7 rows (6 preceding rows and the current row).
- The `PARTITION BY` clause groups the rows by customer_id and product_id so that the running average is calculated separately for each customer and product.
- The `ORDER BY` clause specifies the order in which the rows should be processed, which is by sale_date in this case.



Write SQL queries from natural language

PROMPT STRUCTURE

I want you to act as a data scientist. I am running {PostgreSQL 14/MySQL 8/SQLite 3.4/other versions.}. I have the tables {table_name} which are {table description}. The sales table consists of the columns {column names}. Can you please write a query that finds {required window function}?

Example: Window functions in SQL



I want you to act as a data scientist. I have three PostgreSQL 14 tables "sales" for an e-commerce company. The sales table consists of the columns "customer_id", "product_id", "sale_date", "sale_quantity". Can you please write a query that finds the difference between the sale_quantity of that day and the average sale_quantity?



Certainly! Here's an example query that should do what you're asking for:



sql

Copy code

```
SELECT
  s.sale_date,
  s.sale_quantity,
  AVG(s.sale_quantity) OVER (ORDER BY s.sale_date ROWS BETWEEN 6 PRECEDING
    s.sale_date - AVG(s.sale_quantity) OVER (ORDER BY s.sale_date ROWS B
FROM
  sales s
ORDER BY
  s.sale_date;
```



Data generation workflow

PROMPT STRUCTURE

I want you to act as a data generator in Python. Can you generate a Markdown file that contains {data requirement}. Save the file to {filename}

Example: Generate Markdown



I want you to act as a data generator in python. Can you generate a Markdown file that contains mock employee data containing the columns employee_id, name, department_id, email, join_date, current_salary. Save the file to 'employee.md'



Sure! Here's an example of how you could generate a Markdown file in Python with mock employee data:

```
python Copy code

import pandas as pd

# Generate mock employee data
employee_data = {
    'employee_id': [1001, 1002, 1003, 1004, 1005],
    'name': ['John Doe', 'Jane Smith', 'Bob Johnson', 'Alice Brown', 'Peter'],
    'department_id': [101, 102, 101, 103, 102],
    'email': ['john.doe@example.com', 'jane.smith@example.com', 'bob.johnson',
    'join_date': ['2020-01-01', '2019-05-15', '2021-02-10', '2018-09-01', '2
    'current_salary': [60000, 75000, 50000, 90000, 65000]
}

# Convert to a Pandas DataFrame
df = pd.DataFrame(employee_data)

# Save to Markdown file
with open('employee.md', 'w') as f:
    f.write(df.to_markdown(index=False))
```



Data generation workflow

PROMPT STRUCTURE

I want you to act as a data generator in Python. Can you generate a CSV file that contains {data requirement}. Save the file to {filename}

Example: Generate CSV



I want you to act as a data generator in python. Can you generate a CSV file that contains mock employee data containing the columns employee_id, name, department_id, email, join_date, current_salary. Save the file to 'employee.csv'



Sure! Here's some Python code that generates mock employee data and saves it to a CSV file named 'employee.csv':

python

Copy code

```
import csv
import faker
import random
import datetime

fake = faker.Faker()

# Define the header row for the CSV file
header = ['employee_id', 'name', 'department_id', 'email', 'join_date', 'current_salary']

# Define the number of rows to generate
num_rows = 100

# Define the range of department IDs to choose from
dept_ids = list(range(1, 11))

# Generate mock employee data
employees = []
for i in range(num_rows):
    employee_id = i + 1
    name = fake.name()
    department_id = random.choice(dept_ids)
    email = fake.email()
```



Data generation workflow

PROMPT STRUCTURE

I want you to act as a data generator in Python. Can you generate a JSON file that contains {data requirement}. Save the file to {filename}

Example: Generate JSON



I want you to act as a data generator in python. Can you generate a JSON file that contains mock employee data containing the columns employee_id, name, department_id, email, join_date, current_salary. Save the file to 'employee.json'



Sure, here's an example Python code to generate a JSON file with mock employee data:



```
python Copy code

import json
import random
import datetime

# Generate a list of mock employee data
employees = []
for i in range(1, 101):
    employee_id = i
    name = "Employee " + str(i)
    department_id = random.randint(1, 10)
    email = "employee" + str(i) + "@company.com"
    join_date = datetime.date.today() - datetime.timedelta(days=random.randint(0, 365))
    current_salary = round(random.uniform(30000, 80000), 2)
    employees.append({"employee_id": employee_id,
                     "name": name,
                     "department_id": department_id,
                     "email": email,
                     "join_date": join_date.isoformat(),
                     "current_salary": current_salary})

# Write the list of employee data to a JSON file
with open('employee.json', 'w') as f:
    json.dump(employees, f, indent=4)
```



Data cleaning workflow

PROMPT STRUCTURE

I want you to act as a data scientist programming in Python Pandas. Given a CSV file that contains data of {dataframe name} with the columns {column names} for {dataset context}, write code to clean the data? {Insert requirements for data}



I want you to act as a data scientist programming in Python Pandas. Given a CSV file that contains data of "customers" information with the columns "customer_id", "customer_name", "customer_email", "customer_phone", "customer_address" for an e-commerce company, write code to clean the data? Please remove rows with customer_id, and replace null customer_name with "UNKNOWN".



Certainly! Here's some sample code that should accomplish what you've described:

```
python
import pandas as pd

# Read in the CSV file
df = pd.read_csv('customers.csv')

# Drop rows with missing customer_id
df = df.dropna(subset=['customer_id'])

# Replace null customer_name with "UNKNOWN"
df['customer_name'] = df['customer_name'].fillna('UNKNOWN')

# Write cleaned data to a new CSV file
df.to_csv('cleaned_customers.csv', index=False)
```



Data analysis workflow in pandas

PROMPT STRUCTURE

I want you to act as a data scientist programming in Python Pandas. Given a table {table name} that consists of the columns {column names} can you please write a query that finds {requirement}?

Example: Data Aggregation



I want you to act as a data scientist programming in Python Pandas. Given a table "sales" for an e-commerce company that consists of the columns "customer_id", "product_id", "sale_date", "sale_quantity", can you please write a query that finds the most popular product_id in January?



Sure! Here's the code to find the most popular product_id in January:

```
python
import pandas as pd

# Load the sales data into a pandas dataframe
sales = pd.read_csv('sales.csv')

# Filter the data to only include sales in January
jan_sales = sales[sales['sale_date'].str.startswith('2022-01')]

# Group the sales by product_id and sum the sale_quantity
product_sales = jan_sales.groupby('product_id')['sale_quantity'].sum()

# Find the product_id with the highest total sale_quantity
most_popular_product = product_sales.idxmax()

print(f'The most popular product in January is {most_popular_product}')
```

This code assumes that the date in the 'sale_date' column is in the format 'YYYY-MM-DD'. If it's in a different format, you'll need to adjust the code accordingly.



Data analysis workflow in pandas

PROMPT STRUCTURE

I want you to act as a data scientist programming in Python Pandas. Given a table {table 1 name} that consists of the columns {column names} and another table {table 2 name} with the columns {column names}, please merge the two tables. {Insert additional requirement, if any}

Example: Data Merging



I want you to act as a data scientist programming in Python Pandas. Given a table "sales" for an e-commerce company that consists of the columns "customer_id", "product_id", "sale_date", "sale_quantity", and another table "products" with the columns "product_id", "product_name", please merge the two tables and find the name of the most popular product in January.



Sure! Here's an example code to merge the two tables and find the name of the most popular product in January:

```
python Copy code

import pandas as pd

# Read the sales table
sales = pd.read_csv('sales.csv')

# Read the products table
products = pd.read_csv('products.csv')

# Merge the two tables based on product_id
merged_df = pd.merge(sales, products, on='product_id')

# Filter for sales in January
jan_sales = merged_df[merged_df['sale_date'].str.startswith('2022-01')]

# Group by product_id and sum the sale_quantity
sales_by_product = jan_sales.groupby('product_id').sum('sale_quantity')

# Merge with the products table to get the product name
sales_by_product = pd.merge(sales_by_product, products, on='product_id')

# Sort by sale_quantity and get the name of the most popular product
most_popular = sales_by_product.sort_values('sale_quantity', ascending=False)

print(f'The most popular product in January was {most_popular}.')
```



Data analysis workflow in pandas

PROMPT STRUCTURE

I want you to act as a data scientist programming in Python Pandas. Given a table {table name} that consists of the columns {column names} can you aggregate the {value} by {column} and convert it from long to wide format?

Example: Data Reshaping



I want you to act as a data scientist programming in Python Pandas. Given a table "sales" for an e-commerce company that consists of the columns "customer_id", "product_id", "sale_date", "sale_quantity", and another table "products" with the columns "product_id", "product_name", please merge the two tables and find the name of the most popular product in January.



Sure! Here's an example code to merge the two tables and find the name of the most popular product in January:

```
python Copy code

import pandas as pd

# Read the sales table
sales = pd.read_csv('sales.csv')

# Read the products table
products = pd.read_csv('products.csv')

# Merge the two tables based on product_id
merged_df = pd.merge(sales, products, on='product_id')

# Filter for sales in January
jan_sales = merged_df[merged_df['sale_date'].str.startswith('2022-01')]

# Group by product_id and sum the sale_quantity
sales_by_product = jan_sales.groupby('product_id').sum('sale_quantity')

# Merge with the products table to get the product name
sales_by_product = pd.merge(sales_by_product, products, on='product_id')

# Sort by sale_quantity and get the name of the most popular product
most_popular = sales_by_product.sort_values('sale_quantity', ascending=False)
```



Data generation workflow

PROMPT STRUCTURE

I want you to act as a data generator in R. Can you generate a Markdown file that contains {data requirement}. Save the file to {filename}

Example: Generate Markdown



I want you to act as a data generator in R. Can you generate a Markdown file that contains mock employee data containing the columns employee_id, name, department_id, email, join_date, current_salary. Save the file to 'employee.md'



Sure, here's an example of how to generate mock employee data in R and save it to a Markdown file:



```
{r} Copy code

library(tidyverse)
library(lubridate)

# Define number of employees
n_employees <- 100

# Generate mock employee data
employee_data <- tibble(
  employee_id = 1:n_employees,
  name = sample(c("John", "Jane", "Bob", "Alice"), n_employees, replace = TRUE),
  department_id = sample(c(1, 2, 3), n_employees, replace = TRUE),
  email = paste0(name, sample(c(".smith", ".doe", ".jones"), n_employees, replace = TRUE), ".doe", ".jones"),
  join_date = sample(seq(as.Date("2010/01/01"), as.Date("2022/01/01"), by = "year"), n_employees),
  current_salary = round(runif(n_employees, 30000, 80000), 2)
)

# Convert join_date to a character string in YYYY-MM-DD format
employee_data$join_date <- as.character(employee_data$join_date, format = "%Y-%m-%d")

# Save employee_data to a Markdown file
write.table(employee_data, "employee.md", sep = "|", quote = FALSE, row.names = FALSE)
```



Data generation workflow

PROMPT STRUCTURE

I want you to act as a data generator in R. Can you generate a CSV file that contains {data requirement}. Save the file to {filename}

Example: Generate CSV



I want you to act as a data generator in R. Can you generate a CSV file that contains mock employee data containing the columns employee_id, name, department_id, email, join_date, current_salary. Save the file to 'employee.csv'



Sure, here's an example code to generate mock employee data and save it as a CSV file named 'employee.csv':



```
{r} Copy code

# Load necessary packages
library(stringr)
library(lubridate)

# Set seed for reproducibility
set.seed(123)

# Define number of employees
n_employees <- 100

# Generate employee IDs
employee_id <- sprintf("E%03d", 1:n_employees)

# Generate employee names
first_names <- c("John", "Jane", "Bob", "Alice", "David", "Emily", "Chris",
last_names <- c("Smith", "Johnson", "Brown", "Lee", "Garcia", "Davis", "Jone
name <- paste(sample(first_names, n_employees, replace = TRUE), sample(last_

# Generate department IDs
department_id <- sprintf("D%02d", sample(1:10, n_employees, replace = TRUE))

# Generate employee emails
email <- paste(str_to_lower(substring(name, 1, 1)), last_names, "@company.co
```



Data generation workflow

PROMPT STRUCTURE

I want you to act as a data generator in R. Can you generate a JSON file that contains {data requirement}. Save the file to {filename}

Example: Generate JSON



I want you to act as a data generator in R. Can you generate a JSON file that contains mock employee data containing the columns employee_id, name, department_id, email, join_date, current_salary. Save the file to 'employee.json'



Sure! Here's an example of generating a JSON file with mock employee data using the `jsonlite` package in R:



```
r Copy code

library(jsonlite)

# Set number of employees
num_employees <- 100

# Create mock data frame
employee_data <- data.frame(
  employee_id = 1:num_employees,
  name = replicate(num_employees, paste(sample(LETTERS, 5), collapse="")),
  department_id = sample(1:5, num_employees, replace=TRUE),
  email = replicate(num_employees, paste(sample(letters, 8, replace=TRUE), "
  join_date = sample(seq(as.Date('2015/01/01'), as.Date('2020/12/31'), by="d
  current_salary = sample(50000:150000, num_employees)
)

# Write to JSON file
write_json(employee_data, "employee.json")
```

This will generate a JSON file called `employee.json` with 100 rows of mock employee data. You can adjust the `num\_employees` variable to generate more or fewer rows of data.



Data cleaning workflow

PROMPT STRUCTURE

I want you to act as a data scientist programming in R tidy. You are given the {dataframe name} dataframe containing the columns {column name}. {Insert requirement}



I want you to act as a data scientist programming in R tidy. You are given the 'employee' dataframe containing the columns employee_id, name, department_id, email, join_date, current_salary. Write code that drops rows that have any null values and perform other data cleaning steps.



Sure, here's an example code that drops rows with any null values and performs some additional data cleaning steps:

```
r Copy code

library(dplyr)

# Load the employee dataframe
employee <- read.csv("employee.csv")

# Drop rows with any null values
employee_cleaned <- employee %>% drop_na()

# Clean up the email column by removing whitespace and converting to lowercase
employee_cleaned$email <- tolower(trimws(employee_cleaned$email))

# Clean up the join_date column by converting to date format
employee_cleaned$join_date <- as.Date(employee_cleaned$join_date, format="%Y-%m-%d")

# Sort the dataframe by join_date in ascending order
employee_cleaned <- employee_cleaned %>% arrange(join_date)

# Save the cleaned dataframe to a new CSV file
write.csv(employee_cleaned, "employee_cleaned.csv", row.names=FALSE)
```